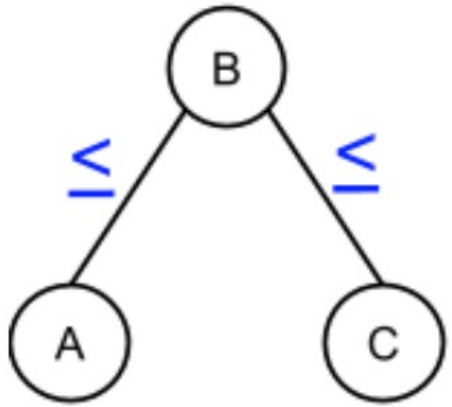
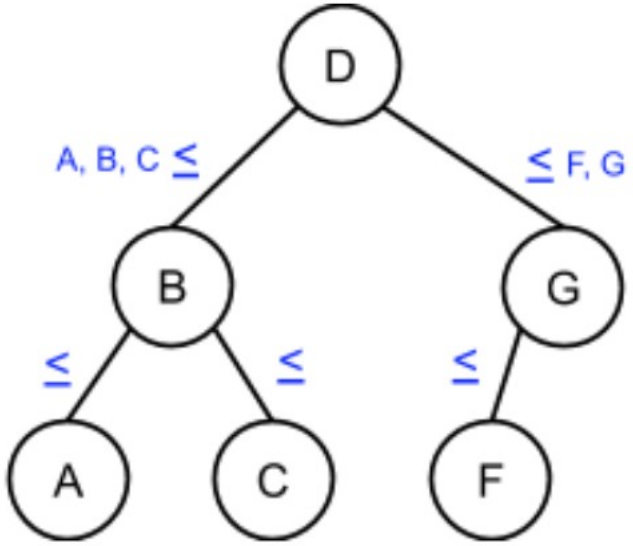
Binary Search Trees

Binary Search Tree

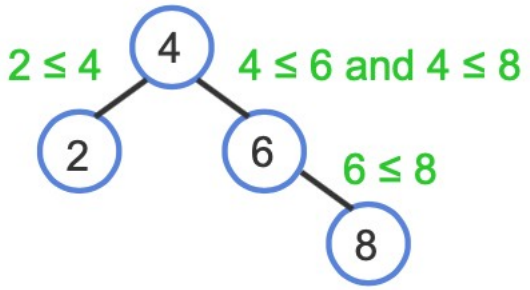




Left Child < Parent

Right Child >= Parent

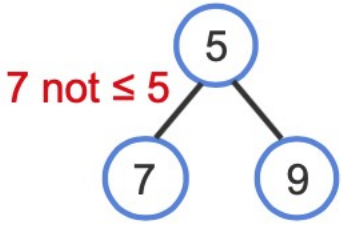
BST –examples



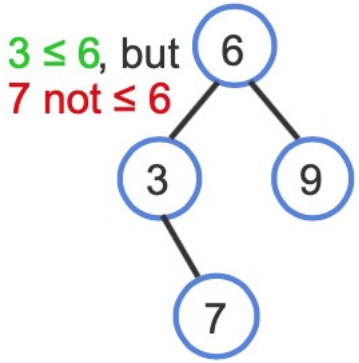
BST ordering property:

Left subtree's keys ≤ node's key

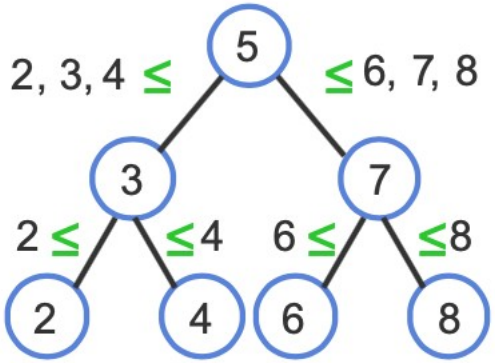
Right subtree's keys ≥ node's key



Not a BST



Not a BST



Valid BST

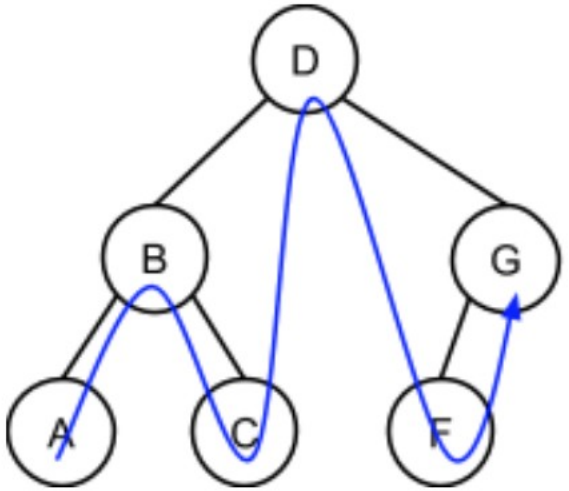
BST – Operations

1. Search
2. Insert
3. Remove

In-order Traversal

BST follow in-order traversal.

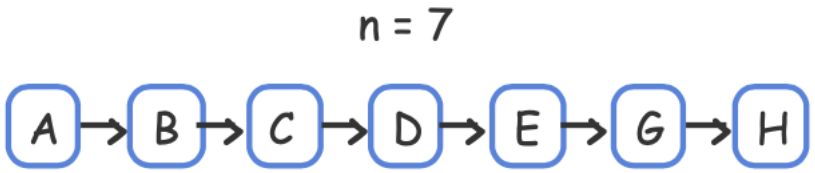
Left first – **node** next – **right** last



BST ordering:
A B C D F G

Successor follows in ordering.
Ex: D's successor is F.

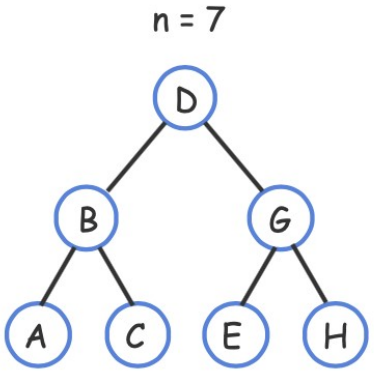
BST Search



node first – **left** next – **right** last

```
if (currentNode→key == desiredKey) {
   return currentNode; // The desired node was found
}
else if (desiredKey < currentNode→key) {
   // Visit left child, repeat
}
else if (desiredKey > currentNode→key) {
   // Visit right child, repeat
}
```

Search for F: 7 comparisons, F not found.

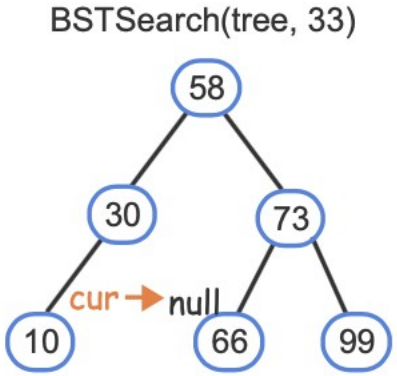


Search for F: 3 comparisons, F not found.

A perfect binary tree has height $\lfloor log_2 N \rfloor$.
A perfect binary tree search is O(H), so O($logN$).

BST Search Algorithm

```
BSTSearch(tree, key) {
   cur = tree→root
   while (cur is not null) {
      if (key == cur→key) {
         return cur // Found
      }
      else if (key < cur→key) {
         cur = cur→left
      }
      else {
         cur = cur→right
      }
   }
   return null // Not found
}
```



BST Insert Algorithm

Insert as left child: If the new node's key is less than the current node, and the current node's left child is null, the algorithm assigns that node's left child with the new node.
Insert as right child: If the new node's key is greater than or equal to the current node, and the current node's right child is null, the algorithm assigns the node's right child with the new node.
Search for insert location: If the left (or right) child is not null, the algorithm assigns the current node with that child and continues searching for a proper insert location.

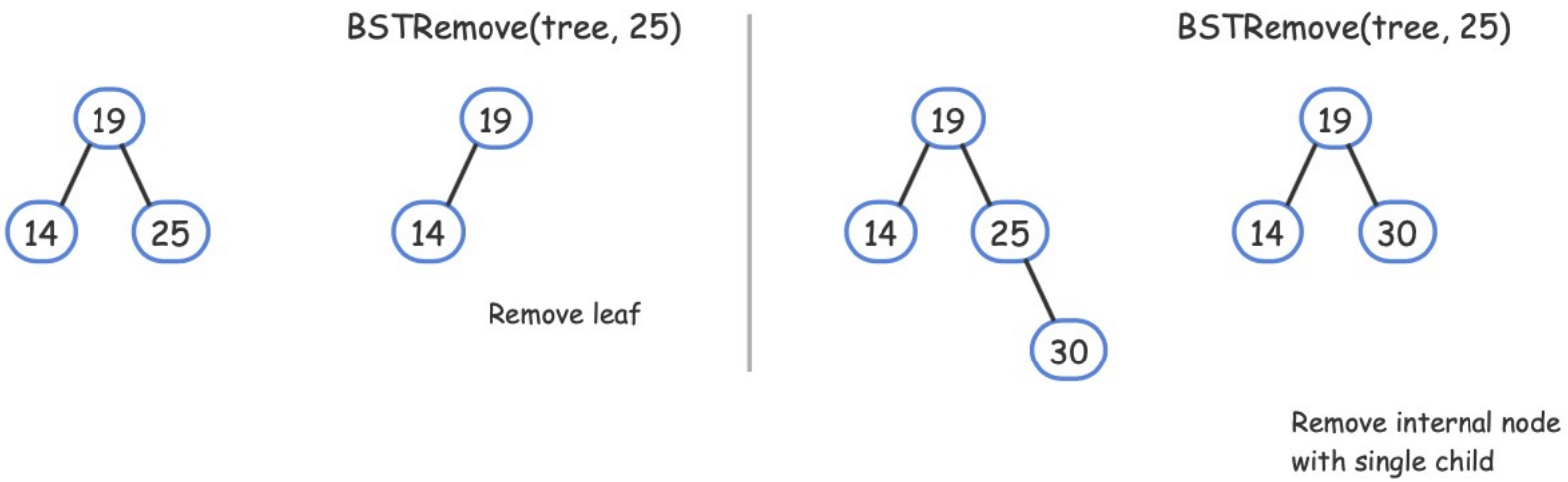
BST Insert Algorithm

```
BSTInsert(tree, node) {
   if (tree→root is null) {
      tree→root = node
   }
   else {
      currentNode = tree→root
      while (currentNode is not null) {
         if (node→key < currentNode→key) {
            if (currentNode→left is null) {
               currentNode→left = node
               currentNode = null
            }
            else {
               currentNode = currentNode→left
            }
         }
         else {
            if (currentNode→right is null) {
               currentNode→right = node
               currentNode = null
            }
            else {
               currentNode = currentNode→right
            }
         }
      }
   }
}
```

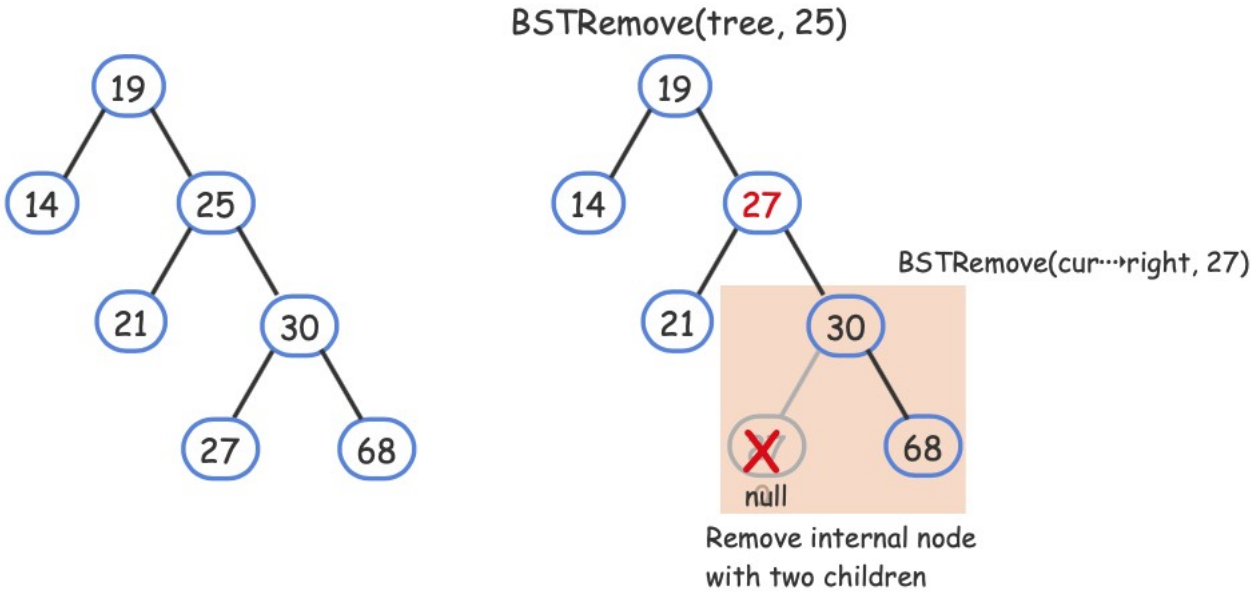
The BST insert algorithm traverses the tree from the root to a leaf node to find the insertion location. One node is visited per level. A BST with N nodes has at least $log_2 N$ levels and at most N levels. Therefore, the runtime complexity of insertion is best case $O(logN)$ and worst case $O(N)$.

BST Remove

Remove a leaf node: If X has a parent (so X is not the root), the parent's left or right child (whichever points to X) is assigned with null. Else, if X was the root, the root pointer is assigned with null, and the BST is now empty.
Remove an internal node with single child: If X has a parent (so X is not the root), the parent's left or right child (whichever points to X) is assigned with X's single child. Else, if X was the root, the root pointer is assigned with X's single child.
Remove an internal node with two children: This case is the hardest. First, the algorithm locates X's successor (the leftmost child of X's right subtree), and copies the successor to X. Then, the algorithm recursively removes the successor from the right subtree.

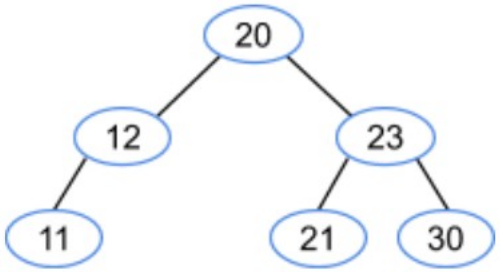


BST Remove (internal node with 2 children)



The BST remove algorithm traverses the tree from the root to find the node to remove. When the node being removed has two children, the node's successor is found and a recursive call is made. One node is visited per level, and in the worst case scenario the tree is traversed twice from the root to a leaf. A BST with N nodes has at least $log_2 N$ levels and at most N levels. So removal's worst case time complexity is $O(logN)$ for a BST with $log_2 N$ levels and worst case $O(N)$ for a tree with N levels.

Text Notation for BST



(20 (12 (11, -), 23 (21, 30)))